

Giáo Án

Phiên bản web: <https://chief-heliotrope-d20.notion.site/Gi-o-n-1c7b9e967cbc80d5996ee123f8aee507?pvs=4>

📌 Lưu ý cho giảng viên:

- **Tái sử dụng mã nguồn Todo App của bài học trước:** Bài học này **tiếp nối trực tiếp** bài học về Redux Toolkit cơ bản. Hãy đảm bảo học viên đang sử dụng mã nguồn Todo App đã xây dựng ở bài học trước.
- **Backend Express.js đã chuẩn bị:** Sử dụng mã nguồn Express.js backend đã cung cấp dưới đây và hướng dẫn học viên chạy backend server trước khi bắt đầu thực hành frontend.

Khởi tạo backend đơn giản

```
// server.js
const express = require('express');
const cors = require('cors');
const app = express();
const port = 5000;

app.use(cors({
  origin: 'http://localhost:3000' // CHÚ Ý: CÓ THỂ THAY ĐỔI CHO PHÙ HỢP VỚI DOMAIN VÀ PORT FRONTEND CỦA BẠN.
}));
app.use(express.json());

let todos = [
  { id: 1, text: 'Học React', completed: false },
  { id: 2, text: 'Làm bài tập Redux', completed: true },
  { id: 3, text: 'Thực hành gọi API', completed: false },
];

app.get('/api/todos', (req, res) => {
  res.json(todos);
});

app.post('/api/todos', (req, res) => {
  const newTodo = { ...req.body, id: Date.now(), completed: false };
  todos.push(newTodo);
  res.status(201).json(newTodo);
});

app.patch('/api/todos/:id', (req, res) => { // Sử dụng PATCH thay vì PUT
  const id = parseInt(req.params.id);
  const todoIndex = todos.findIndex(todo => todo.id === id);
  if (todoIndex > -1) {
    todos[todoIndex] = { ...todos[todoIndex], ...req.body }; // Merge các thuộc tính được gửi lên
    res.json(todos[todoIndex]);
  } else {
    res.status(404).json({ message: 'Todo not found' });
  }
});

app.delete('/api/todos/:id', (req, res) => {
  const id = parseInt(req.params.id);
  todos = todos.filter(todo => todo.id !== id);
  res.status(204).send();
});
```

```
app.listen(port, () => {
  console.log(`Server listening at <http://localhost>:${port}`);
});
```

Hướng dẫn học viên chạy backend:

- **Cài đặt Node.js và npm:** Đảm bảo máy tính đã cài đặt Node.js và npm.
- **Khởi tạo thư mục project mới (nếu chưa có):** Trong thư mục chứa file `server.js`, mở terminal và chạy `npm init -y`.
- **Cài đặt các dependencies:** Chạy `npm install express cors`.
- **Chạy server:** Chạy lệnh `node server.js` hoặc `nodemon server.js`.
- **Kiểm tra:** Mở trình duyệt và truy cập `http://localhost:5000/api/todos`.

Ôn tập Redux Toolkit và Giới thiệu Redux Thunk

1. Mục tiêu

- Học viên củng cố kiến thức về Redux Toolkit từ bài trước.
- Học viên hiểu được vấn đề cần giải quyết khi làm việc với tác vụ bất đồng bộ trong Redux.
- Học viên nắm bắt được khái niệm và vai trò cơ bản của Redux Thunk như một giải pháp.

2. Nội dung

2.1 Ôn lại kiến thức Redux Toolkit đã học

- Giảng viên ôn nhanh lại các kiến thức: Store, Slice, Actions, Reducers, Selectors, `Provider`, `useSelector`, `useDispatch`.
- Review nhanh code Todo App bài trước (cấu trúc Redux Toolkit).

2.2 Giới thiệu Redux Thunk - Xử lý bất đồng bộ

- Tại sao chúng ta cần Redux Thunk?

Hãy tưởng tượng ứng dụng Todo của chúng ta **chưa dùng Redux Thunk**. Chúng ta muốn lấy danh sách todos từ API server và hiển thị lên UI.

- Cách tiếp cận thông thường (VẪN có thể dùng, nhưng có hạn chế):

Bạn có thể gọi API trực tiếp **ngay trong component**, ví dụ trong `useEffect` của `TodoList` component, sau đó dispatch một action để cập nhật Redux store:

```
// CÁCH TIẾP CẬN VẪN DÙNG ĐƯỢC, NHƯNG CÓ HẠN CHẾ (Ví dụ minh họa)
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useDispatch } from 'react-redux';
import { setTodos } from '../redux/slices/todoSlice'; // Giả sử có action creator đồng bộ setTodos

function TodoList() {
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const dispatch = useDispatch();

  useEffect(() => {
    const fetchTodos = async () => {
      try {
        const response = await fetch('/api/todos'); // Gọi API trực tiếp trong component
        dispatch(setTodos(response.data)); // Dispatch action ĐỒNG BỘ để cập nhật Redux store
        setLoading(false);
      } catch (err) {
        setError(err);
        setLoading(false);
      }
    };
    fetchTodos();
  }, []);
}
```

```
    }
  };
  fetchTodos();
}, [dispatch, setTodos]); // Thêm dispatch và setTodos vào dependency array

if (loading) return <p>Loading todos...</p>;
if (error) return <p>Error: {error.message}</p>;

return (
  // ... (render danh sách todos từ Redux store bằng useSelector) ...
  <ul>
    {/* ... (map todos từ Redux state) */}
  </ul>
);
}

export default TodoList;
```

- Tuy nhiên, cách tiếp cận này có những hạn chế và trở nên kém hiệu quả khi ứng dụng phức tạp hơn:
 - Thiếu khả năng tái sử dụng logic.** Logic gọi API hiện đang được viết riêng cho component `TodoList`. Nếu component khác (ví dụ: `TodoDashboard`, `TodoStatistics`) cũng cần lấy danh sách todos từ API, chúng ta sẽ phải **lặp lại toàn bộ logic gọi API** này. Không có cách nào dễ dàng để tái sử dụng logic fetching data này ở nhiều nơi trong ứng dụng.
 - Logic nghiệp vụ (gọi API, xử lý data) vẫn còn trộn lẫn với logic UI (component).** Component `TodoList` vẫn phải chịu trách nhiệm về việc gọi API, xử lý lỗi, và quản lý cập nhật trạng thái.
 - Khó test logic bất đồng bộ một cách độc lập.** Việc test logic gọi API và xử lý lỗi trở nên khó khăn hơn khi nó nằm trực tiếp trong component.
- Redux Thunk giúp chúng ta giải quyết các hạn chế trên bằng cách:**
 - Tăng khả năng tái sử dụng logic nghiệp vụ:** Redux Thunk trở thành các "module" độc lập chứa logic gọi API.

Do đó, chúng ta không cần viết lại logic gọi API lặp lại ở nhiều nơi.

 - Tách biệt hoàn toàn logic nghiệp vụ (gọi API, xử lý data) ra khỏi component.** Component chỉ tập trung vào việc dispatch actions (thunk action creators) và hiển thị UI dựa trên Redux state.
 - Việc test logic nghiệp vụ cũng trở nên dễ dàng hơn.
 - Luồng dữ liệu rõ ràng và dễ theo dõi hơn cho các tác vụ bất đồng bộ.** Mọi thứ liên quan đến việc gọi API và cập nhật state đều được quản lý thông qua actions và reducers, tuân theo luồng dữ liệu Redux.
- Nói ngắn gọn, Redux Thunk giúp:**
 - Logic nghiệp vụ (đặc biệt là bất đồng bộ) được tổ chức tốt hơn, dễ bảo trì và tái sử dụng.
 - Component tập trung vào UI, code gọn gàng hơn.
 - Quản lý trạng thái bất đồng bộ tập trung và nhất quán trong Redux.
 - Code dễ test và mở rộng hơn.

Cấu hình Môi trường

1. Mục tiêu

- Học viên biết cách quản lý cấu hình ứng dụng, đặc biệt là thông tin API endpoint, bằng file `.env`.

2. Nội dung

2.2 Giới thiệu và thao tác với `.env`

- Trong quá trình phát triển ứng dụng, chúng ta thường phải làm việc với nhiều môi trường khác nhau, ví dụ:

- **Môi trường phát triển (Development):** Đây là môi trường trên máy tính cá nhân của bạn, nơi bạn viết code, chạy thử nghiệm, và debug ứng dụng.
 - **Môi trường kiểm thử (Testing/Staging):** Môi trường này giống với môi trường production, nhưng được sử dụng để kiểm thử ứng dụng trước khi release chính thức.
 - **Môi trường sản xuất (Production):** Đây là môi trường mà ứng dụng của bạn chạy chính thức và người dùng cuối sẽ truy cập.
- Mỗi môi trường này thường có **cấu hình khác nhau**. Ví dụ, **địa chỉ API backend** có thể khác nhau giữa môi trường phát triển (`http://localhost:5000/api`) và môi trường production (`https://api.ten-mien-ung-dung-cua-ban.com/api`).
 - **Vấn đề: Tại sao chúng ta không nên "hardcode" cấu hình vào code?**
 - Nếu chúng ta **code cứng (hardcode)** cấu hình này vào code, ví dụ:

```
// KHÔNG NÊN LÀM NHƯ VẬY! (Ví dụ minh họa vấn đề)
const API_BASE_URL = 'http://localhost:5000/api'; // Hardcode URL phát triển

const fetchData = async () => {
  const response = await fetch(`${API_BASE_URL}/todos`);
  // ...
};
```

- Điều này sẽ gây ra nhiều vấn đề:
 - **Khó khăn khi chuyển đổi môi trường:** Mỗi khi bạn muốn chuyển ứng dụng sang môi trường khác (ví dụ từ phát triển lên production), bạn sẽ phải **tìm kiếm và sửa đổi code** ở nhiều nơi để thay đổi cấu hình. Việc này dễ gây nhầm lẫn và sai sót.
 - **Không an toàn cho thông tin nhạy cảm:** Nếu bạn hardcode các thông tin nhạy cảm như trực tiếp vào code và commit lên repository, những thông tin này có thể bị lộ ra ngoài, gây nguy cơ bảo mật.
 - **Khó quản lý cấu hình:** Khi ứng dụng lớn hơn và có nhiều cấu hình hơn, việc quản lý chúng trực tiếp trong code trở nên rất phức tạp và khó theo dõi.
- **Giải pháp: File `.env` - Quản lý cấu hình môi trường:**

Để giải quyết các vấn đề trên, chúng ta sử dụng file `.env` (**environment variables file**). File `.env` là một file văn bản đặc biệt, được đặt ở **thư mục gốc của project**, dùng để **lưu trữ các biến môi trường (environment variables)** cho ứng dụng.

Lợi ích của việc sử dụng `.env`:

- **Tách biệt cấu hình khỏi code:** Cấu hình được lưu trữ riêng trong file `.env`, code của bạn không chứa các giá trị cấu hình cụ thể.
 - **Dễ dàng chuyển đổi môi trường:** Để chuyển ứng dụng sang môi trường khác, bạn chỉ cần thay đổi file `.env` tương ứng, **không cần sửa code**.
 - **Bảo mật thông tin nhạy cảm:** Bạn có thể lưu trữ các thông tin nhạy cảm trong file `.env` và **không commit file `.env` lên repository**.
 - **Quản lý cấu hình tập trung:** Tất cả cấu hình của ứng dụng được quản lý tập trung trong các file `.env`, dễ dàng theo dõi và chỉnh sửa.
-  **Thêm file `.env` vào dự án Todo App:**
 - Tạo file `.env` trong thư mục gốc của project React (cùng cấp với package.json, src, public...), tạo một file mới tên là `.env`. Chú ý dấu chấm `.` ở đầu tên file.
 - Mở file `.env` và thêm các biến môi trường theo cú pháp:

```
VITE_API_BASE_URL=http://localhost:5000/api
```

- **Lưu ý tiền tố `VITE_`:** Đối với các project React được tạo bằng **Vite**, tên biến môi trường phải bắt đầu bằng `VITE_`.
- **Không có dấu ngoặc kép (") hoặc nháy đơn ('):** Giá trị của biến môi trường được viết trực tiếp sau dấu bằng `=`.

Tối ưu Gọi API với Axios Instance

1. Nội dung

- Học viên so sánh và lựa chọn được thư viện gọi API phù hợp (Axios).
- Học viên nắm vững kỹ thuật tạo và sử dụng Axios Instance để tái sử dụng cấu hình API client, giúp code gọn gàng và dễ bảo trì hơn.

2. Nội dung

2.1 Fetch API vs. Axios

Khi chúng ta cần gọi API từ ứng dụng React, có hai lựa chọn phổ biến để thực hiện các HTTP requests: **Fetch API** và **Axios**.

Fetch API

- **Ưu điểm:**
 - **Built-in (có sẵn):** Fetch API là một phần của trình duyệt web hiện đại. Điều này có nghĩa là bạn **không cần phải cài đặt bất kỳ thư viện bên ngoài nào** để sử dụng nó. Bạn có thể bắt đầu gọi API ngay lập tức.
- **Nhược điểm:**
 - **Cú pháp phức tạp hơn một chút:** Cú pháp của Fetch API có thể hơi dài dòng và kém trực quan hơn so với Axios, đặc biệt là khi xử lý các request phức tạp hoặc cấu hình các tùy chọn nâng cao.
 - **Xử lý lỗi HTTP không trực quan:** Fetch API **không tự động bắt lỗi HTTP** (ví dụ: lỗi 404 Not Found, 500 Internal Server Error). Ngay cả khi server trả về mã lỗi, Fetch API vẫn coi request là "thành công" ở mức độ network. Bạn cần phải **kiểm tra thuộc tính `response.ok`** để xác định xem request có thực sự thành công hay không, và tự xử lý các trường hợp lỗi.
 - **Không tự động parse JSON response:** Khi bạn nhận được dữ liệu JSON từ API, Fetch API **không tự động chuyển đổi nó thành object JavaScript**. Bạn cần phải sử dụng phương thức `response.json()` để parse JSON response một cách thủ công.

Axios

- **Ưu điểm:**
 - **Cú pháp đơn giản và dễ sử dụng:** Axios cung cấp một API trực quan và dễ sử dụng hơn Fetch API. Code gọi API với Axios thường ngắn gọn và dễ đọc hơn.
 - **Xử lý lỗi HTTP tốt hơn:** Axios **tự động bắt lỗi HTTP**. Nếu server trả về mã lỗi (ví dụ: 404, 500), Axios sẽ **ném ra một lỗi**, giúp bạn xử lý các trường hợp lỗi một cách trực tiếp và dễ dàng hơn bằng cơ chế try...catch.
 - **Tự động parse JSON response:** Axios **tự động chuyển đổi JSON response thành object JavaScript**. Bạn không cần phải gọi `response.json()` như với Fetch API.
 - **Nhiều tính năng hữu ích khác:** Axios cung cấp nhiều tính năng nâng cao và tiện lợi hơn Fetch API, như:
 - **Interceptors:** Cho phép bạn can thiệp vào request và response trước khi chúng được xử lý (ví dụ: thêm headers chung, xử lý lỗi tập trung).
 - **Hủy bỏ request (Cancellation):** Cho phép bạn hủy bỏ một request đang chờ xử lý.
 - **Bảo vệ chống lại XSRF:** Tự động tích hợp cơ chế bảo vệ chống tấn công Cross-Site Request Forgery (XSRF).
- **Nhược điểm:**
 - **Cần cài đặt thư viện:** Axios là một thư viện bên ngoài, vì vậy bạn cần phải **cài đặt nó vào project của mình** bằng npm hoặc yarn (`npm install axios`) trước khi sử dụng.

2.2 Axios Instance - Tái sử dụng cấu hình

- **Axios Instance:** Tạo custom Axios client cấu hình chung. Dùng instance gọi API, không lặp lại cấu hình.
- **Tạo Axios Instance:**

```
// src/api/axiosClient.js
import axios from 'axios';

const axiosClient = axios.create({
  baseURL: process.env.VITE_API_BASE_URL, // Lấy từ .env
  headers: {
    'Content-Type': 'application/json',
    // Thêm header chung khác nếu cần
  },
});

export default axiosClient;
```

- **Sử dụng Axios Instance:**

```
// Ví dụ sử dụng axiosClient để GET data
import axiosClient from './api/axiosClient';

const fetchData = async () => {
  try {
    const response = await axiosClient.get('/todos'); // Chỉ cần endpoint path
    console.log("Data from API:", response.data);
  } catch (error) {
    console.error("Error fetching data:", error);
  }
};
```

Kết nối Component với API và Redux Thunk

1. Mục tiêu

- Học viên hiểu cách kết hợp `useEffect` hook và Redux Thunk để fetch dữ liệu API khi component mount.
- Học viên nắm được quy trình chuyển đổi các chức năng Todo App (Xem, Thêm, Sửa, Xóa) sang gọi API thông qua Redux Thunk.

2. Nội dung

2.1 Chuyển đổi chức năng "Xem danh sách Todo" sang gọi API

- **Bước 1:** Cập nhật `todoSlice.js` - Thêm Async Thunk `fetchTodos` và reducers quản lý trạng thái fetch:

```
// src/redux/slices/todoSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axiosClient from '../api/axiosClient';

export const fetchTodos = createAsyncThunk(
  'todos/fetchTodos', // Action type: 'todos/fetchTodos'
  async () => {
    const response = await axiosClient.get('/todos');
    return response.data;
  }
);

const todoSlice = createSlice({
  name: 'todos',
  initialState: {
    todos: [],
```



```

    loading: false,
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchTodos.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchTodos.fulfilled, (state, action) => {
        state.loading = false;
        state.todos = action.payload;
      })
      .addCase(fetchTodos.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      });
  },
});

export const {} = todoSlice.actions;
export default todoSlice.reducer;

```

◦ **Giải thích** `'todos/fetchTodos'` :

- Đây là **tiền tố action type**. Redux Toolkit sẽ tự động tạo ra 3 action types đầy đủ dựa trên tiền tố này và lifecycle của promise:
 - **Pending:** `'todos/fetchTodos/pending'` (Báo hiệu request bắt đầu).
 - **Fulfilled:** `'todos/fetchTodos/fulfilled'` (Báo hiệu request thành công).
 - **Rejected:** `'todos/fetchTodos/rejected'` (Báo hiệu request thất bại).
- Action type thường có cấu trúc `'sliceName/actionName'` :
 - `sliceName` : Tên slice (ví dụ: `'todos'`). Giúp nhóm các action liên quan đến một feature/domain lại với nhau.
 - `actionName` : Tên action (ví dụ: `'fetchTodos'`, `'addTodo'`, `'toggleTodo'`). Mô tả hành động.

• **Bước 2:** Cập nhật `TodoList.jsx` :

```

// src/components/ToDoList/ToDoList.jsx
import React, { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchTodos } from '../redux/slices/todoSlice';
import TodoItem from '../TodoItem/ToDoItem';

function ToDoList() {
  const dispatch = useDispatch();
  const todos = useSelector((state) => state.todos.todos);
  const loading = useSelector((state) => state.todos.loading);
  const error = useSelector((state) => state.todos.error);

  useEffect(() => {
    dispatch(fetchTodos());
  }, [dispatch]);

  if (loading) {
    return <p>Loading todos...</p>;
  }
}

```

```
if (error) {
  return <p>Error loading todos: {error}</p>;
}

return (
  <ul>
    {todos.map((todo) => (
      <TodoItem key={todo.id} todo={todo} />
    ))}
  </ul>
);
}

export default TodoList;
```

- **Giảng viên** giải thích `useEffect` hook chạy khi component mount (`[]` dependency), dispatch `fetchTodos` , dùng `useSelector` lấy `todos` , `loading` , `error` và render UI tương ứng.

2.2 Cập nhật tương tự cho các chức năng khác

Giảng viên tham khảo mã nguồn đầy đủ bên dưới để hướng dẫn học viên hoàn thiện chuyển đổi code sang gọi API bằng Redux Thunk:

Cấu trúc folder:

```
your-todo-app/
├── src/
│   ├── api/
│   │   └── axiosClient.js
│   ├── components/
│   │   ├── TodoInput/
│   │   │   └── TodoInput.jsx
│   │   ├── TodoItem/
│   │   │   └── TodoItem.jsx
│   │   └── TodoList/
│   │       └── TodoList.jsx
│   ├── redux/
│   │   ├── slices/
│   │   │   └── todoSlice.js
│   │   └── store.js
│   ├── screens/
│   │   └── Home/
│   │       └── Home.jsx
│   ├── App.jsx
│   └── main.jsx
├── .env
└── .gitignore
```

```
// src/api/axiosClient.js
import axios from 'axios';

const axiosClient = axios.create({
  baseURL: process.env.VITE_API_BASE_URL, // Lấy từ .env
  headers: {
    'Content-Type': 'application/json',
  },
});

export default axiosClient;
```



```
// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
import todoReducer from './slices/todoSlice';

const store = configureStore({
  reducer: {
    todos: todoReducer,
  },
});

export default store;
```

```
// src/redux/slices/todoSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axiosClient from '../api/axiosClient';

export const fetchTodos = createAsyncThunk(
  'todos/fetchTodos',
  async () => {
    const response = await axiosClient.get('/todos');
    return response.data;
  }
);

export const addTodo = createAsyncThunk(
  'todos/addTodo',
  async (todoText) => {
    const response = await axiosClient.post('/todos', { text: todoText });
    return response.data;
  }
);

export const toggleTodo = createAsyncThunk(
  'todos/toggleTodo',
  async (todold, { getState }) => {
    const todo = getState().todos.todos.find(todo => todo.id === todold);
    if (!todo) return;

    const updatedTodo = { completed: !todo.completed };
    await axiosClient.patch(`/todos/${todold}`, updatedTodo);
    return todold;
  }
);

export const deleteTodo = createAsyncThunk(
  'todos/deleteTodo',
  async (todold) => {
    await axiosClient.delete(`/todos/${todold}`);
    return todold;
  }
);

const todoSlice = createSlice({
  name: 'todos',
  initialState: {
    todos: [],
    loading: false,
    error: null,
```

```

},
reducers: {},
extraReducers: (builder) => {
  builder
    .addCase(fetchTodos.pending, (state) => {
      state.loading = true;
      state.error = null;
    })
    .addCase(fetchTodos.fulfilled, (state, action) => {
      state.loading = false;
      state.todos = action.payload;
    })
    .addCase(fetchTodos.rejected, (state, action) => {
      state.loading = false;
      state.error = action.error.message;
    })
    .addCase(addTodo.pending, (state) => {
      state.loading = true;
      state.error = null;
    })
    .addCase(addTodo.fulfilled, (state, action) => {
      state.loading = false;
      state.todos.push(action.payload);
    })
    .addCase(addTodo.rejected, (state, action) => {
      state.loading = false;
      state.error = action.error.message;
    })
    .addCase(toggleTodo.pending, (state) => {
      state.loading = true;
      state.error = null;
    })
    .addCase(toggleTodo.fulfilled, (state, action) => {
      state.loading = false;
      const toggledTodold = action.payload;
      const todo = state.todos.find(todo => todo.id === toggledTodold);
      if (todo) {
        todo.completed = !todo.completed;
      }
    })
    .addCase(toggleTodo.rejected, (state, action) => {
      state.loading = false;
      state.error = action.error.message;
    })
    .addCase(deleteTodo.pending, (state) => {
      state.loading = true;
      state.error = null;
    })
    .addCase(deleteTodo.fulfilled, (state, action) => {
      state.loading = false;
      const deletedTodold = action.payload;
      state.todos = state.todos.filter(todo => todo.id !== deletedTodold);
    })
    .addCase(deleteTodo.rejected, (state, action) => {
      state.loading = false;
      state.error = action.error.message;
    });
},
});

```

```
export const { } = todoSlice.actions;
export default todoSlice.reducer;
```

```
// src/components/TodoInput/TodoInput.jsx
import React, { useState } from 'react';
import { useDispatch } from 'react-redux';
import { addTodo } from '../redux/slices/todoSlice';

function TodoInput() {
  const dispatch = useDispatch();
  const [todoText, setTodoText] = useState('');

  const handleAddTodo = () => {
    if (todoText.trim() === '') {
      alert('Vui lòng nhập nội dung todo!');
      return;
    }
    dispatch(addTodo(todoText));
    setTodoText('');
  };

  return (
    <div>
      <input
        type="text"
        placeholder="Nhập todo mới..."
        value={todoText}
        onChange={(e) => setTodoText(e.target.value)}
      />
      <button onClick={handleAddTodo}>Add Todo</button>
    </div>
  );
}

export default TodoInput;
```

```
// src/components/TodoList/TodoList.jsx
import React, { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchTodos } from '../redux/slices/todoSlice';
import TodoItem from '../TodoItem/TodoItem';

function TodoList() {
  const dispatch = useDispatch();
  const todos = useSelector((state) => state.todos.todos);
  const loading = useSelector((state) => state.todos.loading);
  const error = useSelector((state) => state.todos.error);

  useEffect(() => {
    dispatch(fetchTodos());
  }, [dispatch]);

  if (loading) {
    return <p>Loading todos...</p>;
  }

  if (error) {
    return <p>Error loading todos: {error}</p>;
  }
}
```

```

    }

    return (
      <ul>
        {todos.map((todo) => (
          <TodoItem key={todo.id} todo={todo} />
        ))}
      </ul>
    );
  }
}

export default TodoList;

```

```

// src/components/TodoItem/TodoItem.jsx
import React from 'react';
import { useDispatch } from 'react-redux';
import { toggleTodo, deleteTodo } from '../../redux/slices/todoSlice';

function TodoItem({ todo }) {
  const dispatch = useDispatch();

  const handleToggleTodo = () => {
    dispatch(toggleTodo(todo.id));
  };

  const handleDeleteTodo = () => {
    dispatch(deleteTodo(todo.id));
  };

  return (
    <li>
      <input
        type="checkbox"
        checked={todo.completed}
        onChange={handleToggleTodo}
      />
      <span style={{ textDecoration: todo.completed ? 'line-through' : 'none' }}>
        {todo.text}
      </span>
      <button onClick={handleDeleteTodo}>Delete</button>
    </li>
  );
}

export default TodoItem;

```

```

// src/screens/Home/Home.jsx
import React from 'react';
import TodoInput from '../../components/TodoInput/TodoInput';
import TodoList from '../../components/TodoList/TodoList';

function Home() {
  return (
    <div>
      <h1>Ứng Dụng Todo với Redux Toolkit</h1>
      <TodoInput />
      <TodoList />
    </div>
  );
}

```

```
);  
}  
  
export default Home;
```

```
// src/App.jsx  
import React from 'react';  
import { BrowserRouter, Routes, Route } from 'react-router-dom';  
import Home from './screens/Home/Home';  
  
function App() {  
  return (  
    <BrowserRouter>  
      <Routes>  
        <Route path="/" element={<Home />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}  
  
export default App;
```

```
// src/main.jsx  
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import App from './App';  
import store from './redux/store';  
import { Provider } from 'react-redux';  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <React.StrictMode>  
    <Provider store={store}>  
      <App />  
    </Provider>  
  </React.StrictMode>  
>);
```

.env:

```
VITE_API_BASE_URL=http://localhost:5000/api
```

- 📌 **Lưu ý cho giảng viên:**
 - Cho học viên làm **Bài Tập Cuối Chương**.